Deep Learning Practicals 1 to 5

Complete Question and Answer Guide

Code-related questions, theory questions with point-wise explanations and examples, and a final set of general Deep Learning questions and answers.

Practical	Topic	Main Tools
1	Installation of TensorFlow, Keras, PyTorch, Theano	pip, Jupyter / Colab
2	Feedforward Neural Network (MNIST)	Keras, TensorFlow, SGD
3	Image Classification using CNN (MNIST)	Conv2D, MaxPooling2D, Adam
4	Anomaly Detection using Autoencoder	Keras Functional API, MSE
5	Continuous Bag of Words (CBOW) model	Tokenizer, Embedding layer
6	General Deep Learning Q&A	Theory and interview questions

How to use this guide

- Each practical has: **Aim**, **Code-related Q&A**, and **Theory Q&A**.
- Parameter counts and output shapes are calculated by hand so you can answer viva questions confidently.
- Accuracy values written as 'approx.' will vary slightly on every run because weights start randomly.

Practical 1: Installation of TensorFlow, Keras, PyTorch and Theano

Aim

To install the major Deep Learning frameworks (TensorFlow, Keras, PyTorch, Theano) and verify that each one imports correctly.

Part A: Code-related Questions

Q1. Write the commands used to install the four frameworks.

- `pip install tensorflow` installs TensorFlow (Keras is already bundled inside it).
- `pip install keras` installs standalone Keras (safe even if already present).
- `pip install torch torchvision` installs PyTorch (CPU version) and torchvision for image datasets and models.
- `pip install Theano` installs Theano. If it fails on import with new NumPy, use `pip install pytensor`.
- On Google Colab TensorFlow, Keras and PyTorch are pre-installed; only Theano or PyTensor needs `!pip install`.

Q2. Explain the verification code line by line.

- `import tensorflow as tf`, `import keras`, `import torch` load the libraries; an ImportError means installation failed.
- `tf.__version__`, `keras.__version__`, `torch.__version__` store the installed version string of each library.
- `print(...)` shows the versions; take a screenshot of this output for your journal.
- The `try` block imports Theano. If that raises any exception, the `except` block imports PyTensor, the maintained fork, and prints its version.

Q3. Why is a try/except block used for Theano?

- Theano is discontinued, so it may install but crash on import with modern NumPy or Python.
- try/except lets the program continue instead of stopping with a traceback.
- The fallback to PyTensor shows you know the actively maintained replacement.

Q4. How do you check whether a GPU is available?

- TensorFlow: `tf.config.list_physical_devices('GPU')` returns a list; an empty list means no GPU.
- PyTorch: `torch.cuda.is_available()` returns True or False.

  Example: On Colab choose Runtime > Change runtime type > GPU, then run both commands.

Q5. How do you install a specific version, or check what is installed?

- Specific version: `pip install tensorflow==2.15.0`.
- List installed packages: `pip list` or `pip show tensorflow`.

Part B: Theory Questions

Q6. What is TensorFlow?

- Open-source Deep Learning library developed by Google Brain.
- Works with tensors (multi-dimensional arrays) and computation graphs.
- Runs on CPU, GPU and TPU; supports mobile (TF Lite) and web (TF.js) deployment.

- Used for production systems because of TF Serving and strong tooling (TensorBoard).

Q7. What is Keras?

- High-level API for building neural networks with very little code.
- Since TensorFlow 2.x it is the official high-level API (`tf.keras`); Keras 3 can also run on JAX and PyTorch.
- Two main styles: Sequential (stack of layers) and Functional (graph of layers, used in Practical 4).

  Example: Sequential([Dense(128), Dense(10)]) builds a two-layer network in one line.

Q8. What is PyTorch?

- Open-source library from Meta (Facebook AI Research).
- Uses dynamic computation graphs: the graph is built while code runs, so debugging with normal Python tools is easy.
- Very popular in research and now also in production.

Q9. What is Theano and why was it discontinued?

- One of the first Deep Learning libraries (University of Montreal, 2007).
- It compiled symbolic math expressions into fast C/CUDA code and supported automatic differentiation.
- Development stopped in 2017 because larger frameworks (TensorFlow, PyTorch) took over.
- PyTensor is its maintained fork and keeps the same ideas.

Q10. Compare the four frameworks.

Feature	TensorFlow	Keras	PyTorch	Theano
Developer	Google	Francois Chollet / Google	Meta	MILA (Montreal)
Level	Low and high	High-level API	Low and high	Low-level
Graph type	Static + eager	Depends on backend	Dynamic	Static symbolic
Status	Active	Active	Active	Discontinued (use PyTensor)
Best for	Production, mobile	Quick prototypes, learning	Research, flexibility	Historical / learning

Q11. Static versus dynamic computation graph?

- **Static:** graph is defined first and then executed (classic TensorFlow 1, Theano). Good optimisation, harder debugging.
- **Dynamic:** graph is created on the fly at each run (PyTorch, TF eager mode). Easier debugging, flexible control flow.

  Example: A loop whose length depends on the input sentence length is natural in a dynamic graph.

Q12. Why use a virtual environment or Colab?

- Virtual environment (venv or conda) keeps each project's package versions separate, avoiding conflicts.
- Colab gives free GPU/TPU and pre-installed libraries, so nothing needs to be installed locally.

Practical 2: Feedforward Neural Network using Keras and TensorFlow

Aim

To build, train and evaluate a feedforward (fully connected) neural network on the MNIST handwritten digit dataset using Keras with the SGD optimizer, and plot training loss and accuracy.

Part A: Code-related Questions

Q1. Explain each step of the code.

- **a. Import:** TensorFlow, Matplotlib, MNIST dataset, Sequential model, Dense and Flatten layers, SGD optimizer.
- **b. Load data:** `mnist.load_data()` returns 60,000 training and 10,000 test images, each 28 x 28 grayscale, with labels 0 to 9.
- **Normalize:** dividing by 255 scales pixels from 0-255 to 0-1.
- **c. Architecture:** Flatten (28x28 -> 784) -> Dense 128 ReLU -> Dense 64 ReLU -> Dense 10 Softmax.
- **d. Compile and train:** SGD with learning rate 0.01, loss `sparse_categorical_crossentropy`, metric accuracy; 5 epochs, batch size 32, 10 percent validation split.
- **e. Evaluate:** `model.evaluate` returns loss and accuracy on the unseen test set.
- **f. Plot:** `history.history` stores per-epoch loss and accuracy for both training and validation; two subplots show the curves.

Q2. Why do we normalize the data?

- Small input values (0 to 1) keep gradients stable and make training faster.
- Large raw values (0 to 255) can cause large weight updates and slow or unstable convergence.

Q3. What does Flatten do?

- Converts a 2-D image (28, 28) into a 1-D vector of 784 values so that Dense layers can accept it.
- It has no trainable parameters.

Q4. Calculate the number of parameters of the model.

- Dense(128): 784 x 128 + 128 bias = **100,480**.
- Dense(64): 128 x 64 + 64 = **8,256**.
- Dense(10): 64 x 10 + 10 = **650**.
- Total trainable parameters = **109,386** (what `model.summary()` shows).

Q5. Why sparse_categorical_crossentropy and not categorical_crossentropy?

- Labels are plain integers (for example 7). The 'sparse' version accepts integers directly.
- categorical_crossentropy needs one-hot labels such as [0,0,0,0,0,0,0,1,0,0].

Q6. How many training steps happen per epoch?

- validation_split=0.1 uses 54,000 images for training and 6,000 for validation.
- Steps per epoch = 54,000 / 32 = 1,687.5, rounded up to **1,688**.

Q7. What is the difference between epoch, batch size and iteration?

- **Epoch:** one full pass over the training data.
- **Batch size:** number of samples processed before one weight update.
- **Iteration:** one weight update (one batch).

  Example: 1,000 samples with batch size 100 gives 10 iterations per epoch.

Q8. How do you read the loss and accuracy plots?

- Both training and validation loss falling together means the model is learning well.
- Training loss falling but validation loss rising means **overfitting**.
- Both losses high and flat means **underfitting** (increase epochs, layers or learning rate).
- Expected result: test accuracy of roughly 94 to 96 percent after 5 epochs with SGD (approx.).

Q9. How can accuracy be improved?

- Use Adam instead of SGD, train more epochs, add Dropout, add more neurons or layers, or use a CNN (Practical 3).

Part B: Theory Questions

Q10. What is a feedforward neural network?

- Information moves only forward: input layer -> hidden layers -> output layer, with no loops.
- Every neuron computes a weighted sum of its inputs plus a bias, then applies an activation function: y = f(w1x1 + w2x2 + ... + b).
- Also called Multi-Layer Perceptron (MLP) when it has one or more hidden layers.

Q11. Explain the activation functions used.

- **ReLU:** f(x) = max(0, x). Fast, reduces vanishing gradient, most common in hidden layers.
- **Softmax:** converts 10 raw scores into probabilities that add up to 1; the largest is the predicted class.

  Example: Scores [2.0, 1.0, 0.1] become probabilities about [0.66, 0.24, 0.10]. Predicted class is the first one.

Q12. What is backpropagation?

- Algorithm that computes the gradient of the loss with respect to every weight using the chain rule, moving from output layer back to input layer.
- Steps: forward pass -> compute loss -> backward pass (gradients) -> update weights with the optimizer.

Q13. What is Stochastic Gradient Descent (SGD)?

- Weight update rule: w = w - learning_rate x gradient.
- Computes gradient on a small batch instead of the full dataset, so it is faster and the noise helps escape poor minima.
- Learning rate 0.01 means each step moves the weight 1 percent of the gradient.

Q14. What is the cross-entropy loss?

- Measures the distance between the true distribution and predicted probabilities: L = -log(p of the correct class).
- If the model gives 0.9 to the correct class, loss is small (0.105); if it gives 0.1, loss is large (2.30).

Practical 3: Image Classification Model using CNN

Aim

To load and preprocess image data, define a Convolutional Neural Network, train it, and estimate its performance on MNIST digits.

Part A: Code-related Questions

Q1. Explain each step of the code.

- **a. Load and preprocess:** load MNIST, convert to float32, divide by 255, reshape from (28,28) to (28,28,1) because Conv2D needs a channel dimension (1 = grayscale). The first 10 images are displayed with labels.
- **b. Architecture:** Conv2D(32) -> MaxPool -> Conv2D(64) -> MaxPool -> Flatten -> Dense(128) -> Dense(10, softmax).
- **c. Training:** Adam optimizer, sparse categorical cross-entropy, 5 epochs, batch size 128, 10 percent validation.
- **d. Evaluation:** `model.evaluate` on test data; `model.predict(x_test[:1])` gives 10 probabilities and `argmax()` picks the digit; the image is shown with predicted and actual labels.

Q2. Calculate the output shape and parameters of every layer.

Layer	Output shape	Parameters (calculation)
Conv2D 32, 3x3	(26, 26, 32)	3 x 3 x 1 x 32 + 32 = 320
MaxPooling 2x2	(13, 13, 32)	0
Conv2D 64, 3x3	(11, 11, 64)	3 x 3 x 32 x 64 + 64 = 18,496
MaxPooling 2x2	(5, 5, 64)	0 (11/2 = 5, floor)
Flatten	(1600)	0 (5 x 5 x 64)
Dense 128	(128)	1600 x 128 + 128 = 204,928
Dense 10	(10)	128 x 10 + 10 = 1,290
Total		**225,034**

Q3. Why is the reshape to (28, 28, 1) needed?

- Conv2D expects 4-D input: (batch, height, width, channels).
- MNIST is stored as (60000, 28, 28), so the channel axis is added. A colour image would have 3 channels (RGB).

Q4. Why does a CNN give better accuracy than the feedforward network of Practical 2?

- It keeps the 2-D spatial structure, whereas Flatten at the start of an FFNN destroys it.
- Filters share weights, so far fewer parameters are needed to detect edges and shapes anywhere in the image.
- Expected test accuracy is roughly 98 to 99 percent (approx.) against about 95 percent for the FFNN.

Q5. Why Adam optimizer here?

- Adam combines momentum and adaptive learning rates per parameter, so it converges faster and needs less tuning than plain SGD.

Q6. What does model.predict return?

- An array of shape (1, 10) containing the softmax probability of each digit.

- `argmax()` returns the index of the highest probability, which is the predicted digit.

Part B: Theory Questions

Q7. What is a Convolutional Neural Network?

- Neural network specialised for grid-like data such as images.
- Built from convolution layers (feature extraction), pooling layers (down-sampling) and dense layers (classification).
- Early layers learn edges, middle layers learn shapes, deep layers learn object parts.

Q8. What is convolution?

- A small filter (kernel), for example 3 x 3, slides over the image; at each position the filter values are multiplied with the image patch and summed to give one output pixel.
- The output is a **feature map**. 32 filters give 32 feature maps.

  Example: A vertical-edge filter produces large values where the image changes from dark to bright horizontally.

Q9. What are stride and padding? Give the output size formula.

- **Stride:** number of pixels the filter moves each step (default 1).
- **Padding:** adding border pixels. 'valid' = no padding (output shrinks), 'same' = zero padding (size preserved).
- Output size = (n - f + 2p) / s + 1, where n is input size, f filter size, p padding, s stride.
- Here: (28 - 3 + 0) / 1 + 1 = 26.

Q10. What is pooling?

- Down-sampling that reduces height and width, so there are fewer parameters and some translation invariance.
- **Max pooling** keeps the maximum of each window; **average pooling** keeps the mean.

  Example: Window [[1,3],[2,8]] gives 8 with max pooling and 3.5 with average pooling.

Q11. What are parameter sharing and local connectivity?

- The same filter weights are reused at every location (parameter sharing).
- Each neuron looks only at a small local region (local connectivity), not the whole image.

Q12. Name popular CNN architectures.

- LeNet-5 (1998, digits), AlexNet (2012), VGG (2014, small 3x3 filters), GoogLeNet/Inception, ResNet (2015, skip connections), EfficientNet, MobileNet (light models).

Practical 4: Anomaly Detection using Autoencoder

Aim

To build an autoencoder that learns to compress and reconstruct images, and to use the reconstruction error to detect anomalies.

Part A: Code-related Questions

Q1. Explain each step of the code.

- **a. Libraries:** NumPy, TensorFlow, Matplotlib, Keras Model, Input and Dense.
- **b. Data:** load MNIST (labels ignored, shown as _), scale to 0-1, flatten each image to 784 values.
- **c. Encoder:** 784 -> 128 -> 64 -> 32. The 32-neuron layer is the **latent (bottleneck)** representation.
- **d. Decoder:** 32 -> 64 -> 128 -> 784 with sigmoid output so pixels stay in 0 to 1.
- **e. Compile:** Adam optimizer, MSE loss, MAE as metric. Training uses `fit(x_train, x_train)` because input and target are the same.
- **Anomaly step:** reconstruct test images, compute per-image mean squared error, set threshold at the 95th percentile, and mark images above it as anomalies.

Q2. Calculate parameters of the autoencoder.

Layer	Calculation	Parameters
Dense 128	784 x 128 + 128	100,480
Dense 64	128 x 64 + 64	8,256
Dense 32 (latent)	64 x 32 + 32	2,080
Dense 64	32 x 64 + 64	2,112
Dense 128	64 x 128 + 128	8,320
Dense 784	128 x 784 + 784	101,136
Total		**222,384**

Q3. Why is the input also used as the target in fit()?

- The goal is reconstruction: output should equal input.
- It is unsupervised (self-supervised): no labels are needed.

Q4. Why sigmoid in the output layer and MSE as loss?

- Pixel values are between 0 and 1, which matches sigmoid's output range.
- MSE measures the average squared pixel difference between original and reconstruction.

Q5. How many anomalies will be found and why?

- Test set has 10,000 images. Threshold is the 95th percentile of errors, so exactly the top 5 percent, about **500 images**, exceed it.
- Note: this means the code always flags 5 percent by definition. It shows the method but does not prove those are truly abnormal.

Q6. How would you make this a real anomaly detector?

- Train only on normal data (for example only digit 1, or digits 0 to 8).
- Then test with normal and abnormal samples (digit 9). The abnormal ones give higher reconstruction error.

- Choose the threshold from validation data, for example mean + 3 x standard deviation of normal errors, then check precision and recall.

Q7. What does the compression ratio tell us?

- 784 values squeezed to 32 gives a ratio of 784 / 32 = **24.5 : 1**.
- The network is forced to keep only the most important features.

Part B: Theory Questions

Q8. What is an autoencoder?

- Neural network trained to copy its input to its output through a narrow bottleneck.
- Two parts: **encoder** (compresses to latent code) and **decoder** (rebuilds the input from the code).
- Used for dimensionality reduction, denoising, anomaly detection, pre-training and image compression.

Q9. How does an autoencoder detect anomalies?

- It learns to rebuild only the patterns seen during training (normal data).
- For an unfamiliar (anomalous) input it reconstructs poorly, giving a high reconstruction error.
- If error > threshold then the sample is an anomaly.

  Example: Credit card fraud: train on normal transactions; a fraudulent transaction has a large reconstruction error.

Q10. Types of autoencoders?

- **Undercomplete:** bottleneck smaller than input (used here).
- **Sparse:** penalty forces few active neurons.
- **Denoising:** input is corrupted with noise, target is the clean image.
- **Variational (VAE):** learns a probability distribution in latent space; can generate new data.
- **Contractive:** penalises sensitivity to small input changes.
- **Convolutional:** uses Conv layers; better for images.

Q11. Why must the bottleneck be small?

- If the hidden layer were as large as the input, the network could simply copy the input (identity function) and learn nothing useful.

Q12. Autoencoder versus PCA?

- PCA is a linear method; an autoencoder with non-linear activations learns non-linear compressions.
- A linear autoencoder with MSE loss gives the same subspace as PCA.

Q13. Why use the Functional API here instead of Sequential?

- Functional API explicitly connects layers like `Dense(...)(input_layer)`, and allows separate encoder and decoder models, multiple inputs and outputs, and shared layers.

Practical 5: Continuous Bag of Words (CBOW) Model

Aim

To prepare text data, generate context-target training pairs and train a CBOW model that predicts a target word from its surrounding context words.

Part A: Code-related Questions

Q1. Explain each step of the code.

- **a. Data preparation:** a small text is tokenized with Keras `Tokenizer`. `word_index` maps each word to an integer; `vocab_size = len(word_index) + 1` (the +1 reserves index 0 for padding).
- `texts_to_sequences` converts the text into a list of integers (21 tokens).
- **b. Training data:** window size 2. For each position i, the context is two words before and two after (4 words) and the target is the centre word.
- **c. Model:** Embedding(vocab_size, 50) turns each word id into a 50-dimensional vector; GlobalAveragePooling1D averages the 4 context vectors; Dense(vocab_size, softmax) predicts the target word. Trained for 100 epochs.
- **d. Prediction:** context words are converted to ids, passed to `predict`, `argmax` gives the id with highest probability, which is mapped back to the word.

Q2. How many training samples are generated and how?

- The text has 21 tokens, the loop runs i = 2 to 18 (range(2, 19)), so **17 samples**.
- Example: for 'deep learning is a part' the context is [deep, learning, a, part] and the target is 'is'.
- Shape of X = (17, 4), shape of y = (17,).

Q3. What is the vocabulary size here?

- There are 12 unique words, so `vocab_size` = 12 + 1 = **13**.
- Embedding parameters = 13 x 50 = 650; Dense parameters = 50 x 13 + 13 = 663.

Q4. Why use GlobalAveragePooling1D?

- The context has 4 word vectors. CBOW treats them as a 'bag' (order ignored) and averages them into one 50-dim vector.
- That single vector is the input to the softmax layer.

Q5. Why is the result of the final prediction not always correct or reproducible?

- The dataset is tiny (17 samples) and weights start randomly, so the output can differ between runs.
- The test context ['deep','is','part','machine'] does not exactly occur in the text, so the model generalises from what it learned.
- A real CBOW model needs a large corpus (millions of words).

Q6. The code gives an error about input_length. What to do?

- In newer Keras 3 versions the `input_length` argument of Embedding has been removed or deprecated.
- Fix: use `Embedding(vocab_size, 50)` and, if needed, call `model.build(input_shape=(None, 4))` or pass `Input(shape=(4,))` first.

Part B: Theory Questions

Q7. What is a word embedding?

- A dense vector (for example 50 numbers) representing a word so that words with similar meaning have similar vectors.
- Learned from data, unlike one-hot vectors which are sparse and carry no meaning.

  Example: Vectors of 'king' and 'queen' are close; king - man + woman is close to queen.

Q8. What is Word2Vec?

- A family of models (Mikolov, Google, 2013) that learn word embeddings from text.
- Has two architectures: **CBOW** and **Skip-gram**.

Q9. CBOW versus Skip-gram?

Point	CBOW	Skip-gram
Input -> Output	Context words -> centre word	Centre word -> context words
Speed	Faster to train	Slower
Works best for	Frequent words, small windows	Rare words, small datasets
Example	[the, cat, on, mat] -> sat	sat -> [the, cat, on, mat]

Q10. What is the window size?

- Number of words considered on each side of the target. Window size 2 means 2 left and 2 right (4 context words).
- Small windows capture syntactic similarity; larger windows capture topical similarity.

Q11. Why is the softmax output layer expensive for large vocabularies?

- Softmax computes scores for every word (vocabulary may be 100,000 or more) at every training step.
- Solutions: **negative sampling** and **hierarchical softmax**.

Q12. Applications of word embeddings?

- Sentiment analysis, machine translation, search and recommendation, chatbots, document similarity, and as input to RNN/LSTM/Transformer models.

Q13. One-hot encoding versus embedding?

- One-hot: vector length = vocabulary size, only one 1, no similarity information, memory heavy.
- Embedding: small dense vector, captures meaning, learned by training.

General Deep Learning Questions and Answers

1. Difference between AI, Machine Learning and Deep Learning?

- **AI:** machines showing intelligent behaviour (widest field).
- **ML:** subset of AI where systems learn from data without explicit rules.
- **DL:** subset of ML using multi-layer neural networks that learn features automatically from raw data.

  Example: Spam filter with hand-made features is ML; a CNN that learns from raw pixels is DL.

2. What is an artificial neuron?

- Computes output = activation(sum of weight x input + bias).
- Weights show importance of inputs, the bias shifts the result and the activation adds non-linearity.

3. Why do we need activation functions?

- Without them any number of layers collapses to a single linear function.
- Non-linear activations let networks learn complex patterns.

4. Name common activation functions.

- **Sigmoid:** output 0 to 1, used for binary output; suffers vanishing gradient.
- **Tanh:** output -1 to 1, zero-centred.
- **ReLU:** max(0,x); default for hidden layers; can cause 'dying ReLU'.
- **Leaky ReLU:** small slope for negative values, fixes dying ReLU.
- **Softmax:** multi-class probabilities.

5. What is the vanishing gradient problem?

- In deep networks gradients become extremely small as they move backward, so early layers stop learning.
- Caused by sigmoid/tanh. Solutions: ReLU, batch normalization, residual connections, LSTM, good initialization.

6. What is the exploding gradient problem?

- Gradients become very large, making weights unstable (NaN loss).
- Solutions: gradient clipping, smaller learning rate, proper initialization.

7. Types of gradient descent?

- **Batch:** whole dataset per update (stable, slow).
- **Stochastic:** one sample per update (fast, noisy).
- **Mini-batch:** small batches, e.g. 32 (best compromise, most used).

8. Explain popular optimizers.

- **SGD with momentum:** adds a fraction of the previous update to speed movement along consistent directions.
- **RMSprop:** divides learning rate by a running average of recent gradient magnitudes.
- **Adam:** momentum + RMSprop; usually the default choice.

9. What is the learning rate?

- Step size of weight updates. Too high: loss jumps or diverges. Too low: very slow training.
- Use schedules, decay or adaptive optimizers.

10. Common loss functions and when to use them?

- **MSE / MAE:** regression and autoencoder reconstruction.
- **Binary cross-entropy:** two-class problems (sigmoid output).

- **Categorical / sparse categorical cross-entropy:** multi-class (softmax output).

11. What is overfitting and how can it be reduced?

- Model memorises training data and fails on new data (high training accuracy, low validation accuracy).
- Remedies: more data, data augmentation, Dropout, L1/L2 regularization, early stopping, smaller model, batch normalization.

12. What is underfitting?

- Model is too simple to learn the pattern; both training and test errors are high.
- Remedies: bigger model, more epochs, better features, lower regularization.

13. What is Dropout?

- Randomly switches off a fraction of neurons (for example 20 to 50 percent) in each training step.
- Forces the network not to depend on single neurons, so it acts as regularization. Disabled during testing.

14. What is Batch Normalization?

- Normalizes the output of a layer to mean 0 and variance 1 for each mini-batch, then scales and shifts it with learnable parameters.
- Benefits: faster and more stable training, allows higher learning rates, mild regularization.

15. Explain the bias-variance trade-off.

- **High bias:** underfitting. **High variance:** overfitting.
- Goal is a model complex enough to capture patterns but simple enough to generalize.

16. What is weight initialization and why is it important?

- Starting values of weights. Zeros make all neurons identical (symmetry problem).
- **Xavier/Glorot** (for sigmoid/tanh) and **He** (for ReLU) keep signal variance stable across layers.

17. What is an RNN? What are its limitations?

- Network with a loop that passes a hidden state from one time step to the next, so it handles sequences (text, speech, time series).
- Limitations: vanishing gradients, hard to learn long-range dependencies, slow because sequential.

18. What are LSTM and GRU?

- **LSTM:** has a cell state and three gates (input, forget, output) to remember or forget information over long sequences.
- **GRU:** simplified version with two gates (update, reset); fewer parameters, often similar accuracy.

19. What is the Transformer and attention mechanism?

- Attention lets the model focus on the most relevant parts of the input while processing each element.
- Transformers use self-attention, process all words in parallel and form the basis of BERT, GPT and Claude.

20. What is a GAN?

- Generative Adversarial Network: a **generator** creates fake data and a **discriminator** tries to tell real from fake.
- They train against each other until the fakes look real. Used for image synthesis and data augmentation.

21. What is transfer learning?

- Reusing a model pre-trained on a large dataset (such as ImageNet) and fine-tuning it on a smaller new dataset.
- Saves time and data. Examples: ResNet, VGG, BERT.

22. Classification metrics: precision, recall, F1?

- **Accuracy** = correct / total.
- **Precision** = TP / (TP + FP): of predicted positives, how many are right.
- **Recall** = TP / (TP + FN): of real positives, how many were found.
- **F1** = 2 x P x R / (P + R). Use it when classes are imbalanced (such as anomaly detection).

23. What is a confusion matrix?

- Table of actual versus predicted classes showing True Positives, False Positives, True Negatives and False Negatives.

24. Sigmoid versus Softmax?

- Sigmoid gives one independent probability (binary or multi-label).
- Softmax gives a probability distribution over mutually exclusive classes that sums to 1.

25. What is data augmentation?

- Creating modified copies of training data (rotate, flip, crop, brightness change) to increase diversity and reduce overfitting.

26. What is early stopping?

- Stop training when validation loss stops improving for a set number of epochs (patience), and restore the best weights.
- Keras: `EarlyStopping(monitor='val_loss', patience=3)`.

27. Sequential API versus Functional API in Keras?

- Sequential: simple linear stack of layers.
- Functional: arbitrary graphs, multiple inputs/outputs, skip connections, shared layers.

28. How do you save and load a Keras model?

- `model.save('model.keras')` stores architecture, weights and optimizer state.
- `tf.keras.models.load_model('model.keras')` reloads it.

29. Why are GPUs used for Deep Learning?

- Neural networks are mostly matrix multiplications; GPUs have thousands of cores that perform them in parallel, so training is many times faster than on a CPU.

30. What are hyperparameters? Give examples.

- Settings chosen before training (not learned): learning rate, batch size, epochs, number of layers/neurons, dropout rate, optimizer, kernel size.
- Tuned with grid search, random search or tools like Keras Tuner.

31. Difference between a parameter and a hyperparameter?

- Parameters (weights and biases) are learned from data. Hyperparameters are set by the programmer.

32. What is the difference between CNN and RNN?

- CNN: grid data (images), spatial patterns, parallel. RNN: sequences, temporal patterns, step by step.

Quick Revision Cheat Sheet

Task	Last layer activation	Loss	Typical optimizer
Binary classification	Sigmoid (1 unit)	binary_crossentropy	Adam
Multi-class (integer labels)	Softmax	sparse_categorical_crossentropy	Adam / SGD
Multi-class (one-hot labels)	Softmax	categorical_crossentropy	Adam / SGD

Task	Last layer activation	Loss	Typical optimizer
Regression	Linear	mse / mae	Adam
Autoencoder (pixels 0-1)	Sigmoid	mse / binary_crossentropy	Adam